

Document:	P1241
Date:	October 8, 2018
Audience:	SG1
Authors:	Lee Howes < lw@fb.com > Eric Niebler < eniebler@fb.com > Lewis Baker < lbaker@fb.com >

In support of merging coroutines into C++20

Summary

We believe that we should move forward with merging the Coroutines TS into the IS for C++20. While there is some possibility that the Core Coroutines [P1063](#) work may show some advantages in some areas it is far from clear, and it will take a long time to reach the same level of confidence over the same set of use cases that the TS already shows.

Facebook's experience

Coroutines are increasingly important to Facebook's long term planning, and likely to the standard as a whole.

We are supporting work on an implementation of coroutines in [GCC](#), improvements to the implementation in LLVM both in terms of optimisations and bug fixes as they come along, developing library types and implementing production libraries based on the TS definition. Lewis Baker has significant experience of implementing functionality using the coroutines TS definitions in the [cppcoro](#) library and is using this experience to build a variety of coroutine support types in the [folly coro](#) library.

We have had experience with the other programming languages we use of transitioning millions of lines of code to the `async / await` style and have become convinced that this improves programmer productivity and code safety significantly. There is enormous advantage in the way libraries that return `Awaitable` types force callers to `await` them, turning codebases naturally into highly asynchronous code structures without significant effort. In addition, it leads to code structures that are asynchronous but safer than the alternatives - it is both harder to make mistakes around lifetime of references, and easier to see those mistakes during code review or bug hunting.

Compared to library-based fibers code we see a different set of advantages around code safety. With the coroutines TS we can enforce in the type system how execution contexts propagate through a call graph, have tight control over where tasks run and what happens when they complete, and relatively easily audit code for misuse of synchronisation primitives. Making the information available in the type system also makes AST-based auditing and code transformation a much more powerful and less error-prone tool.

While there are known gaps in the standard, particularly around return value optimisation and waiter-guaranteed allocation elision especially across virtual function call boundaries, we see a path to filling these gaps that we are comfortable with and that need not affect our implementation strategy. Indeed, [P1063](#) admits that these issues can be largely fixed later.

The concerns expressed by Core Coroutines

The concerns expressed are valid, but we believe fixable in the TS and at the same time not necessarily addressing the issues we see.

- **Syntactic concerns such as missing `co_await` for expected types:** We do not hold a strong opinion either way on this.
- **Guaranteed heap elision:** Core coroutines makes heap elision explicit, which is important for the error handling use case. We believe that heap elision for such a self-contained synchronous operation works well in the TS as it is. Further, the bigger concern we see is not that a given coroutine needs control over how it is allocated, but that the caller needs to be able to guarantee that a coroutine it awaits on has its allocation elided, and that this must happen across ABI boundaries. This is important because a given coroutine might have to allocate in case the caller wishes to detach the `Awaitable` from the `co_await`, but when detaching is not done the caller should be able to rely on heap allocation not happening. This call-site-dependent control is, in our understanding, not addressed, but is important to efficiently support fiber->coroutine rewrites.
- **Lack of reference parameter safety:** Capturing reference parameters in a coroutine is a risk. In practice, though, we don't believe that the Core Coroutines proposal actually makes a meaningful safety improvement, and we would have to rely on correctness checking and code review to make this safe anyway. A function that takes parameters by reference in both approaches has to be treated safely by the caller. What we gain with core coroutines is the fairly limited additional safety that the author of a coroutine has to think about how the parameters end up in the coroutine lambda capture list. We will need to look at best practices for this writing coroutines in practice, but believe we would have to check for reference capture with tooling anyway and that tooling could equally see the hidden capture.
- **API complexity:** The API is complex, but it is far from clear that the same feature set can be achieved without a similarly complex API. The work to fully understand this is non-trivial and will take considerable time.

A call to move forward

It has been said frequently that the best is the enemy of the good. At the present time, the flaws the core coroutines proposal fixes align with a specific use case, and one that we are comfortable with in the TS as it is, and it does not yet address the issues we do see. We could go down the path of waiting, and hoping that that proposal is fleshed out enough to be convincing, is used to implement everything that has been implemented using the TS functionality and shown to be on parity in all other areas, as well as fixing the flaws. This is a long road, and one that may still not take us anywhere.

The community needs to be able to run and not walk in the direction of coroutines, with the huge usability and performance improvements they bring. We can fight over libraries, and that is fine. In particular, we can implement libraries independently until we agree on a reasonable subset. We will be in a true mess if we fight over libraries based on differing fundamentals.

As one example, we believe that the Networking TS in its current form does not optimally support coroutines. We can change this, but if we are to propose that modification we need to know what coroutine language core we are building on. Landing the language feature gives us that fundamental primitives so that we can investigate fully supporting coroutines throughout the standard library in C++23.

For that reason, we should merge the coroutines TS. We should commit to a path, allow the library experimentation on a known core, and move briskly in the direction of improving our asynchronous code through coroutines, and we believe improving our error handling code as well. At Facebook we have decided that we cannot wait and we will simply risk a rewrite if necessary. At the same time, we would prefer to avoid that rewrite risk and our implementation experience makes us happy with the current direction.